

— HANDS-ON WORKSHOP

RxJS

From callback hell to one data stream you can filter, delay and cancel — with diagrams and live demos, step by step.

Callback hell before, one stream after

✗ Without RxJS

```
input.addEventListener('input', () => {
  clearTimeout(timer);
  timer = setTimeout(() => {
    fetch('/api?q=' + input.value)
      .then(res => res.json())
      .then(data => { /* a stale reply can arrive
late */ });
  }, 400);
});
```

You cancel the old timer by hand, manage timers yourself, and have no guarantee a stale response won't land after a fresh one.

✓ With RxJS

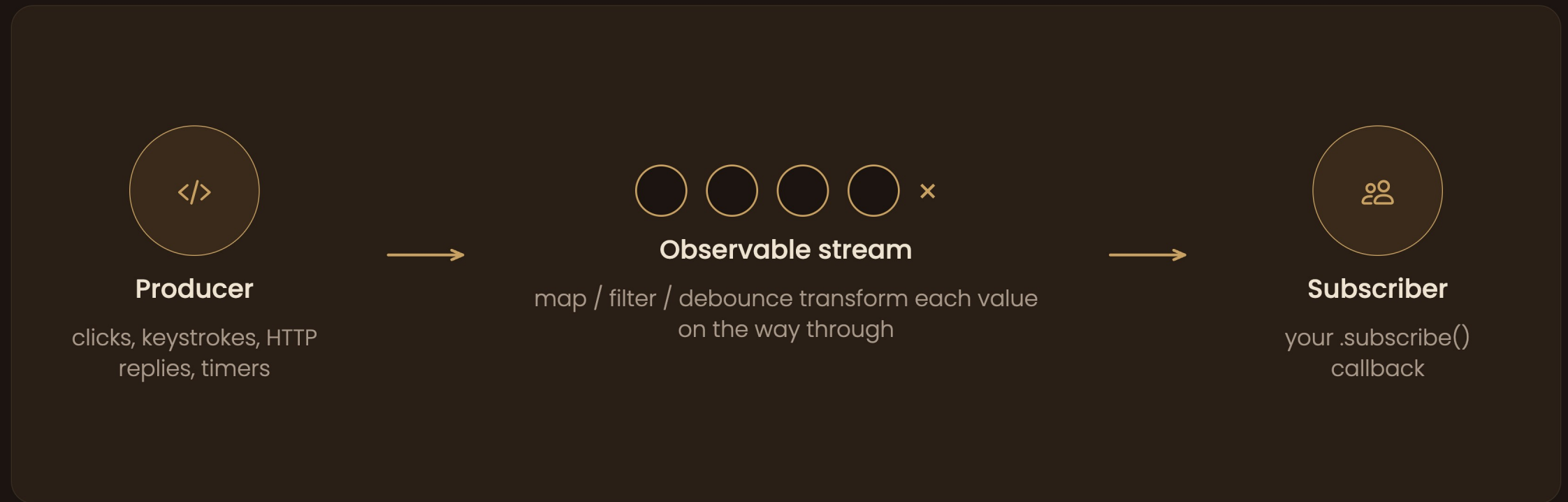
```
searchTerm$.pipe(
  debounceTime(400),
  distinctUntilChanged(),
  switchMap(q => api.search(q))
).subscribe(results => ...);
```

`switchMap` cancels any stale request automatically — order is always guaranteed, and the whole logic lives in one readable place.

02 · THE CORE IDEA

What is an Observable?

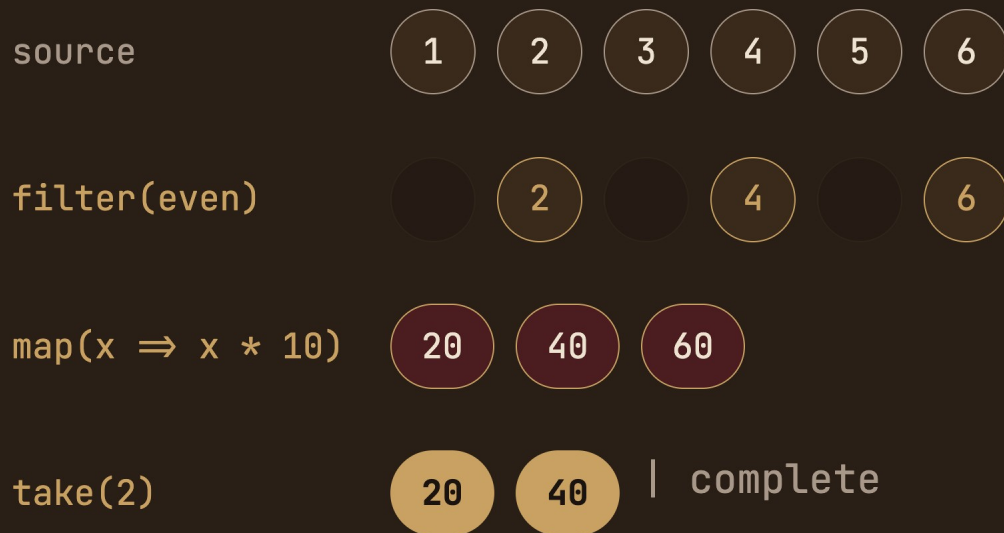
A factory for values that arrive over time. It does nothing until you call **.subscribe()** — like a Netflix show that streams nothing until you hit Play.



03 · OPERATORS

map and filter reshape the stream, mid-flight

Operators live inside `.pipe()`. Each one takes the value coming from above and passes a new one down — like stations on a factory line.



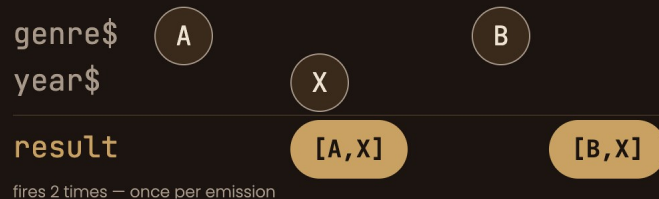
05 · COMBINING STREAMS

combineLatest vs forkJoin

Both wait on more than one Observable at once — the difference is **when** they emit.

combineLatest

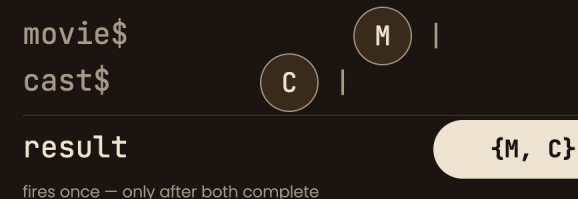
Emits every time ANY source emits, pairing it with the latest value from every other source. Good for live-combining independent inputs, like two filter dropdowns.



```
combineLatest([genre$, year$])  
  .subscribe(([g, y]) => filter(g, y));
```

forkJoin

Waits for every source to COMPLETE, then emits ONCE with the last value of each — like Promise.all. Perfect for "load these 3 endpoints, then render."

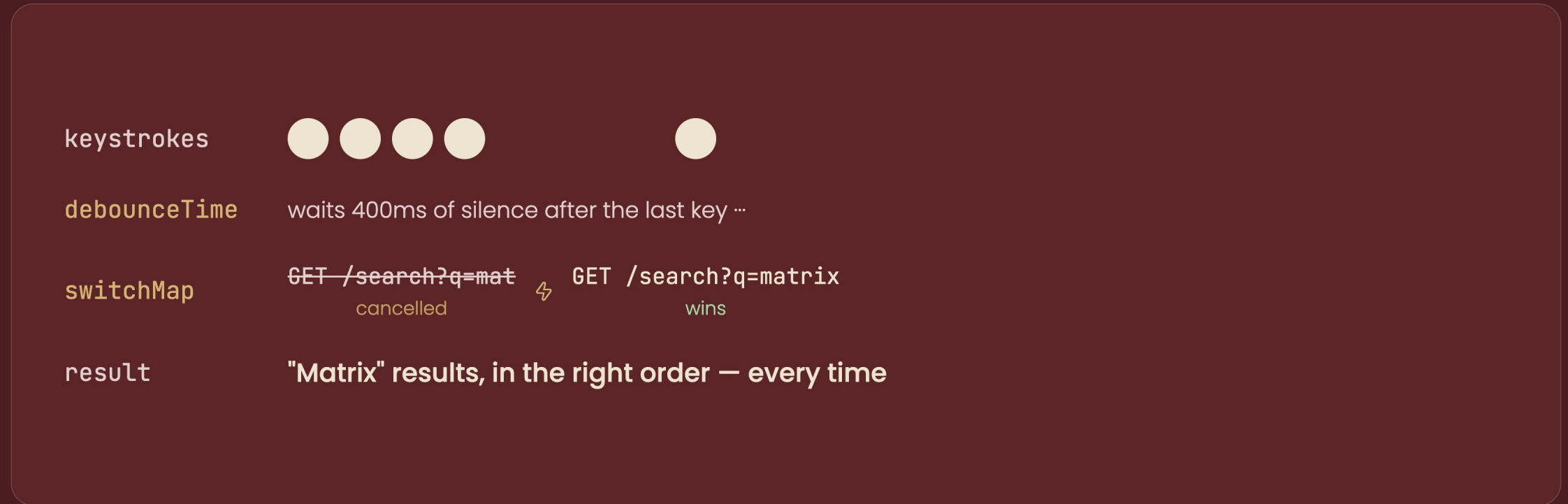


```
forkJoin({ movie: movie$, cast: cast$ })  
  .subscribe({ movie, cast } => render());
```

06 · A REAL EXAMPLE

A search box that cancels stale requests

debounceTime(400) waits for a pause in typing. **distinctUntilChanged()** skips repeats. **switchMap** drops the previous request the moment a new one starts.



debounceTime, one keystroke at a time

It doesn't "slow down" typing — it **waits for silence**. Every keystroke resets its internal timer; only when nobody types for the given duration does a value get through.

⊗ Without debounceTime

keystrokes **c** **a** **t**

requests ~~GET ?q=c~~ → ~~GET ?q=ca~~ → GET ?q=cat

Three requests for one search. Wasted calls, and a race: **?q=c** or **?q=ca** can reply *after ?q=cat* and flash the wrong results on screen.

✓ With debounceTime(400)

keystrokes **c** **a** **t**

timer 400ms of silence after the last key ...

requests GET ?q=cat

One request, fired 400ms after the **last** keystroke — not the first, not each one. Typing "cat" fast never triggers "c" or "ca" at all.

`searchTerm$.pipe(debounceTime(400))` — same idea whether the source is a search box, a resize event, or a window-scroll stream.

distinctUntilChanged only looks one step back

It skips a value **only if it's identical to the one immediately before it** — not to "any value seen so far." That's a common mix-up worth seeing on a real stream.

source\$

emits



distinctUntilChanged()



The second **A** passes through again after **B** — it only repeated the value two steps back, not the value immediately before it. `distinctUntilChanged` has no memory beyond the last emission.

Continuing the search box (page 06)

```
user types: "cat" → deletes to "ca" → retypes "cat"
without distinctUntilChanged: 2× GET /search?q=cat
with distinctUntilChanged: 1× GET /search?q=cat
```

Where it goes in the pipe

```
searchTerm$.pipe(
  debounceTime(400),
  distinctUntilChanged(), // after debounce
  switchMap(q ⇒ api.search(q))
).subscribe(...);
```

subscribe() you forget to close never stops

Subscribing to a live source (interval, router events) keeps that subscription alive in memory — even after the component that created it is gone.

- Open page → subscribe() → running
- Close page → nobody unsubscribed → still running
- Open it again → a second copy now runs alongside

...and so on: memory leak

The classic fix

```
private sub = new Subscription();

ngOnInit() {
  this.sub.add(source$.subscribe(...));
}

ngOnDestroy() {
  this.sub.unsubscribe();
}
```

Let Angular subscribe and unsubscribe for you

The **| async** pipe subscribes automatically when the element appears, and unsubscribes automatically when it's removed. No leak is even possible.

✗ Manual

```
movies: Movie[] = [];  
private sub!: Subscription;  
  
ngOnInit() {  
  this.sub = this.movies$.subscribe(  
    m => this.movies = m);  
}  
  
ngOnDestroy() {  
  this.sub.unsubscribe(); // easy to forget  
}
```

✓ async pipe

```
movies$ = this.api.getMovies();  
  
// in the template, that's it:  
@for (m of movies$ | async; track m.id) {  
  ...  
}
```

No subscribe. No unsubscribe. No leak.

The operators you'll reach for daily

Operator	What it does	Example
map	Transforms every value into a new one	<code>map(x => x * 2)</code>
filter	Keeps only values that pass a condition	<code>filter(x => x > 0)</code>
debounceTime	Waits for a quiet period before emitting	<code>debounceTime(400)</code>
distinctUntilChanged	Skips a value that repeats the last one	<code>distinctUntilChanged()</code>
switchMap	Switches to a new Observable, cancelling the old one — perfect for search	<code>switchMap(q => api.search(q))</code>
mergeMap	Runs every request in parallel, no cancelling	<code>mergeMap(id => api.get(id))</code>
combineLatest	Emits whenever any source emits, paired with the others' latest values	<code>combineLatest([a\$, b\$])</code>
forkJoin	Waits for every source to complete, then emits once — like <code>Promise.all</code>	<code>forkJoin({ a: a\$, b: b\$ })</code>
takeUntil	Closes the stream when another Observable fires — ideal in <code>ngOnDestroy</code>	<code>takeUntil(this.destroy\$)</code>
catchError	Catches a stream error and returns a safe fallback	<code>catchError(() => of([]))</code>

— YOUR TURN

Now go build with it

Open the **RxJS** tab in the Nightfall Stream app: a live Observable, an operator playground, a real debounced search, a memory-leak demo you can trigger yourself, and an async pipe example side by side with the manual version.

 Observable & operators

 debounce + switchMap

 memory leaks

 async pipe